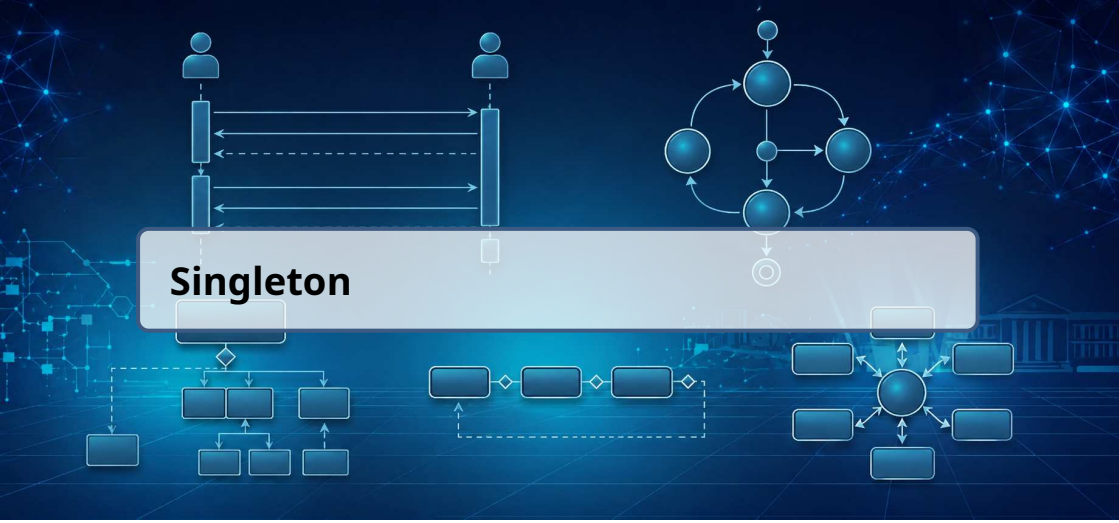


15.1 Introducción

El patrón Singleton es un patrón de diseño creacional que garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a esa instancia. Este patrón restringe la instanciación de una clase a un solo objeto, lo cual es útil cuando se necesita exactamente un objeto para coordinar acciones en todo el sistema.

El concepto central del Singleton es que la clase misma es responsable de mantener su única instancia. La clase debe asegurarse de que no se puedan crear instancias adicionales y debe proporcionar una forma de acceder a la instancia única. Este control se logra haciendo privado el constructor y proporcionando un método estático que retorna la instancia única.

El patrón Singleton fue documentado por el Gang of Four en su libro “Design Patterns: Elements of Reusable Object-Oriented Software” (1994) como uno de los patrones creacionales fundamentales. Aunque es uno de los patrones más conocidos, también es considerado controvertido por algunos desarrolladores debido a que puede introducir acoplamiento global si se abusa de su uso.

15.2 Necesidad y casos de uso

El patrón Singleton surge de la necesidad de controlar el acceso a recursos compartidos o de garantizar que ciertas operaciones se realicen sobre un

único objeto en todo el sistema. Sin este patrón, múltiples instancias podrían causar inconsistencias, desperdicio de recursos o comportamiento impredecible.

La necesidad fundamental que resuelve este patrón es la de coordinar acciones a través de un único objeto. En muchos sistemas, tener múltiples instancias de ciertos objetos no solo es innecesario, sino que puede ser perjudicial. Por ejemplo, tener múltiples gestores de configuración podría llevar a que diferentes partes del sistema trabajen con configuraciones inconsistentes. Similarmente, múltiples instancias de un logger podrían resultar en mensajes desordenados o conflictos al escribir en el mismo archivo.

15.2.1 Casos de uso típicos

Gestión de conexiones a bases de datos: Una aplicación necesita una única conexión o pool de conexiones a la base de datos. Crear múltiples instancias del gestor de conexiones podría resultar en un uso ineficiente de recursos del servidor de base de datos, límites de conexiones excedidos y problemas de sincronización. El Singleton garantiza que todos los componentes de la aplicación utilicen el mismo gestor de conexiones, optimizando el uso de recursos y facilitando el control de transacciones.

Gestión de configuración: Un objeto que mantiene la configuración de la aplicación debe ser único para garantizar que todos los componentes del sistema accedan a la misma configuración y que los cambios se reflejen en toda la aplicación. Si cada módulo creara su propia instancia de configuración, podría leer diferentes versiones del archivo de configuración o mantener valores inconsistentes en memoria, llevando a comportamientos impredecibles.

Logger o sistema de registro: Un sistema de logging debe ser único para garantizar que todos los mensajes se escriban en el mismo archivo o destino, manteniendo el orden cronológico y evitando conflictos de acceso. Múltiples instancias de logger podrían intentar escribir simultáneamente en el mismo archivo, causando corrupción de datos o pérdida de mensajes. Además, un logger centralizado facilita la configuración de niveles de log y destinos de salida.

Gestión de caché: Una caché en memoria debe ser única para que todos los componentes del sistema compartan los mismos datos cacheados y se maximi-

ce el beneficio del cacheo. Si diferentes partes de la aplicación mantuvieran cachés separadas, se duplicaría el uso de memoria y se perdería la eficiencia que proporciona compartir datos cacheados. Un Singleton de caché asegura que todos los componentes consulten y actualicen la misma caché.

Administrador de recursos: Gestores de impresoras, spoolers, o administradores de hilos que deben coordinar el acceso a recursos limitados del sistema. Por ejemplo, un administrador de pool de hilos debe ser único para evitar la creación excesiva de hilos y coordinar eficientemente las tareas concurrentes. Del mismo modo, un gestor de impresión debe serializar los trabajos de impresión de toda la aplicación.

Registro de objetos: Un registro centralizado que mantiene referencias a objetos importantes del sistema, permitiendo el acceso global a estos objetos sin pasar referencias a través de múltiples capas de la aplicación.

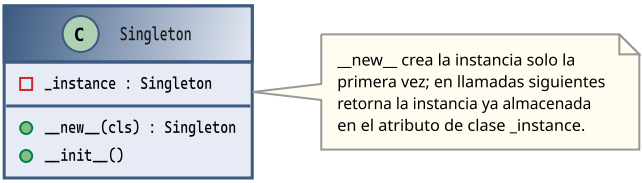
Contadores globales: Generadores de identificadores únicos o contadores que deben garantizar la unicidad de los valores generados en todo el sistema, como generadores de IDs para objetos o números de factura.

15.3 Implementación según GoF

La implementación clásica del patrón Singleton según GoF utiliza un atributo de clase privado para almacenar la única instancia y controla su creación a través del constructor. En Python, esto se logra sobrescribiendo el método especial `__new__`, que es el responsable de crear nuevas instancias de una clase.

El método `__new__` se ejecuta antes que `__init__` y es el encargado de crear y retornar el objeto. Al sobrescribirlo, podemos controlar la creación de instancias y garantizar que solo se cree una. La estrategia consiste en almacenar la primera instancia creada en un atributo de clase y retornar siempre esa misma instancia en llamadas subsecuentes.

El siguiente diagrama de clases resume la estructura de esta implementación, tal como la describe GoF: un atributo de clase estático que retiene la única instancia y un método de clase (`__new__`) que la crea o la retorna:



15.3.1 Implementación básica

```python
class Singleton:
    # Variable de clase para almacenar la única instancia
    _instance = None

    def __new__(cls):
        """
        Sobrescribe el método __new__ para controlar la creación
        de instancias.
        """
        if cls._instance is None:
            # Crear la instancia usando el __new__ de la clase padre
            cls._instance = super(Singleton, cls).__new__(cls)
        return cls._instance

    def __init__(self):
        """
        El __init__ se llama cada vez que se intenta crear
        un objeto, pero __new__ garantiza que siempre sea el mismo.
        """
        pass

# Uso del Singleton
s1 = Singleton()
s2 = Singleton()

print(s1 is s2)  # True - Ambas variables referencian el mismo objeto
print(id(s1), id(s2))  # Muestran el mismo identificador
```

Es importante notar que aunque `__new__` garantiza que solo exista una instancia, el método `__init__` se ejecuta cada vez que se intenta crear un objeto. Esto puede ser problemático si `__init__` realiza inicializaciones costosas o modifica el estado del objeto. Para evitar este problema, se puede agregar una bandera que indique si el objeto ya fue inicializado:

```python
class Singleton:
    _instance = None
    _initialized = False

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(Singleton, cls).__new__(cls)
```

```
        return cls._instance

    def __init__(self):
        # Solo inicializar la primera vez
        if not Singleton._initialized:
            self.data = []
            Singleton._initialized = True
            print("Singleton inicializado")

s1 = Singleton()  # Imprime: Singleton inicializado
s2 = Singleton()  # No imprime nada
s1.data.append(42)
print(s2.data)  # [42] - Comparten el mismo estado
```

15.3.2 Ejemplo con conexión a base de datos

Un caso práctico muy común es la gestión de conexiones a bases de datos. En el siguiente ejemplo se muestra un Singleton que mantiene una única conexión:

```
class DatabaseConnection:
    _instance = None
    _connection_string = None
    _initialized = False

    def __new__(cls, connection_string=None):
        if cls._instance is None:
            cls._instance = super(DatabaseConnection, cls).__new__(cls)
        return cls._instance

    def __init__(self, connection_string=None):
        # Solo inicializar en la primera creación
        if not DatabaseConnection._initialized:
            if connection_string is None:
                raise ValueError("Debe proporcionar connection_string")
            self._connection_string = connection_string
            DatabaseConnection._initialized = True
            print(f"Conexión establecida a: {connection_string}")

    def get_connection_string(self):
        return self._connection_string

    def execute_query(self, query):
        """Simula la ejecución de una query"""
        print(f"Ejecutando en {self._connection_string}: {query}")
        return f"Resultado de: {query}"

    def close(self):
        """Cierra la conexión"""
        print(f"Cerrando conexión a {self._connection_string}")

# Primera creación - establece la conexión
db1 = DatabaseConnection("postgresql://localhost:5432/mydb")

# Segunda creación - retorna la misma instancia
# El connection_string se ignora
```

```
db2 = DatabaseConnection("mysql:// localhost:3306/otherdb")

print(db1 is db2)  # True
print(db1.get_connection_string())
# postgresql:// localhost:5432/mydb

db1.execute_query("SELECT * FROM users")
db2.execute_query("INSERT INTO logs VALUES ('test')")
# Ambas queries se ejecutan en la misma conexión
```

15.3.3 Ejemplo con sistema de configuración

Otro caso común es un sistema de configuración centralizado:

```
import json
from pathlib import Path

class Configuration:
    _instance = None
    _initialized = False

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(Configuration, cls).__new__(cls)
        return cls._instance

    def __init__(self):
        if not Configuration._initialized:
            self.settings = {}
            Configuration._initialized = True

    def load_from_file(self, filepath):
        """Carga la configuración desde un archivo JSON"""
        try:
            with open(filepath, 'r') as f:
                self.settings = json.load(f)
            print(f"Configuración cargada desde {filepath}")
        except FileNotFoundError:
            print(f"Archivo {filepath} no encontrado")

    def get(self, key, default=None):
        """Obtiene un valor de configuración"""
        return self.settings.get(key, default)

    def set(self, key, value):
        """Establece un valor de configuración"""
        self.settings[key] = value

    def save_to_file(self, filepath):
        """Guarda la configuración en un archivo JSON"""
        with open(filepath, 'w') as f:
            json.dump(self.settings, f, indent=4)
        print(f"Configuración guardada en {filepath}")

# Uso en diferentes módulos de la aplicación
# Módulo A
config_a = Configuration()
```

```
config_a.set('database_host', 'localhost')
config_a.set('database_port', 5432)
config_a.set('debug', True)

# Módulo B - obtiene la misma instancia
config_b = Configuration()
print(config_b.get('database_host'))  # localhost
print(config_b.get('debug'))  # True

# Ambas referencias apuntan al mismo objeto
print(config_a is config_b)  # True
```

15.3.4 Ejemplo con logger

Un sistema de logging es otro candidato ideal para Singleton:

```
from datetime import datetime
from pathlib import Path

class Logger:
    _instance = None
    _initialized = False

    # Niveles de log
    DEBUG = 0
    INFO = 1
    WARNING = 2
    ERROR = 3
    CRITICAL = 4

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(Logger, cls).__new__(cls)
        return cls._instance

    def __init__(self):
        if not Logger._initialized:
            self.log_file = None
            self.log_level = Logger.INFO
            self.logs = []
            Logger._initialized = True

    def configure(self, log_file=None, log_level=INFO):
        """Configura el logger"""
        self.log_file = log_file
        self.log_level = log_level

    def _write_log(self, level, message):
        """Escribe un mensaje de log"""
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        level_names = ['DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL']
        log_entry = f"[{timestamp}] {level_names[level]}: {message}"

        self.logs.append(log_entry)

        if self.log_file:
            with open(self.log_file, 'a') as f:
                f.write(log_entry + '\n')
```

```
        else:
            print(log_entry)

    def debug(self, message):
        if self.log_level ≤ Logger.DEBUG:
            self._write_log(Logger.DEBUG, message)

    def info(self, message):
        if self.log_level ≤ Logger.INFO:
            self._write_log(Logger.INFO, message)

    def warning(self, message):
        if self.log_level ≤ Logger.WARNING:
            self._write_log(Logger.WARNING, message)

    def error(self, message):
        if self.log_level ≤ Logger.ERROR:
            self._write_log(Logger.ERROR, message)

    def critical(self, message):
        if self.log_level ≤ Logger.CRITICAL:
            self._write_log(Logger.CRITICAL, message)

# Uso desde diferentes partes de la aplicación
# Módulo de autenticación
logger1 = Logger()
logger1.configure(log_level=Logger.DEBUG)
logger1.info("Usuario intentando iniciar sesión")
logger1.debug("Validando credenciales")

# Módulo de base de datos
logger2 = Logger()
logger2.warning("Conexión lenta a la base de datos")
logger2.error("Fallo en la query")

# Mismo logger en ambos casos
print(logger1 is logger2)  # True
print(len(logger1.logs))  # Muestra todos los logs acumulados
```

15.4 Implementación simplificada en Python

Python ofrece varias alternativas para implementar el patrón Singleton de manera más simple y pythónica, aprovechando características particulares del lenguaje.

15.4.1 Usando un decorador

Una forma elegante y reutilizable de implementar Singleton es mediante un decorador que puede aplicarse a cualquier clase:

```python
def singleton(cls):
    """
    Decorador que convierte una clase en Singleton
    """
    instances = {}

    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]

    return get_instance

@singleton
class Logger:
    def __init__(self):
        self.logs = []

    def log(self, message):
        self.logs.append(message)
        print(f"LOG: {message}")

    def show_logs(self):
        return self.logs

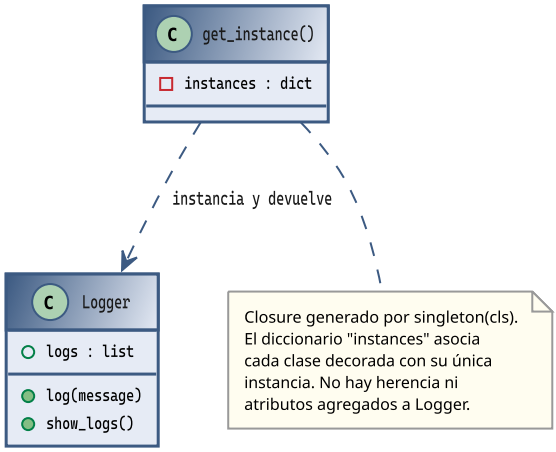
# Uso
logger1 = Logger()
logger1.log("Primer mensaje")

logger2 = Logger()
logger2.log("Segundo mensaje")

print(logger1 is logger2)  # True
print(logger1.show_logs())
# ['Primer mensaje', 'Segundo mensaje']
```

La ventaja de este enfoque es que el decorador es reutilizable y no requiere modificar la estructura interna de la clase. Cualquier clase puede convertirse en Singleton simplemente aplicando el decorador. Además, el decorador mantiene un diccionario de instancias, lo que permite tener múltiples Singletons diferentes si se aplica a varias clases.

A diferencia de la implementación GoF, aquí no existe una relación de herencia ni un atributo de clase dentro de `Logger`: el control de la instancia única queda completamente fuera de la clase, encapsulado en la función `get_instance` generada por el decorador (un *closure* que retiene el diccionario `instances`). El siguiente diagrama complementario ilustra esta diferencia estructural:



Una versión mejorada del decorador que preserva metadatos de la clase original:

```python
from functools import wraps

def singleton(cls):
    """
    Decorador Singleton que preserva metadatos de la clase
    """
    instances = {}

    @wraps(cls)
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]

    # Preservar atributos de clase
    get_instance.__name__ = cls.__name__
    get_instance.__doc__ = cls.__doc__

    return get_instance

@singleton
class Cache:
    """Sistema de caché en memoria"""
    def __init__(self):
        self.data = {}

    def set(self, key, value):
        self.data[key] = value

    def get(self, key, default=None):
        return self.data.get(key, default)

    def clear(self):
```

```
        self.data.clear()

# Uso
cache1 = Cache()
cache1.set("usuario", "Juan")

cache2 = Cache()
print(cache2.get("usuario"))  # Juan
print(Cache.__doc__)  # Sistema de caché en memoria
```

15.4.2 Usando clases abstractas (ABC)

Otra alternativa es emplear una clase base abstracta para estandarizar la forma en que las clases hijas implementan el comportamiento Singleton. La clase abstracta centraliza la lógica de creación única y expone un punto de inicialización para que cada subclase defina su configuración específica.

La idea es:

- Interceptar la creación en `__new__` para garantizar una única instancia por subclase.
- Ofrecer un método abstracto `initialize(...)` que se ejecute solo una vez (la primera vez).
- Permitir que `__init__` sea idempotente delegando en un helper que asegura la inicialización única.

```
from abc import ABC, abstractmethod

class SingletonABC(ABC):
    """Clase base abstracta que implementa Singleton por subclase."""
    _instances = {}
    _initialized = {}

    def __new__(cls, *args, **kwargs):
        if cls is SingletonABC:
            raise TypeError("SingletonABC es abstracta y no debe instanciarse")
        if cls not in cls._instances:
            instance = super().__new__(cls)
            cls._instances[cls] = instance
            cls._initialized[cls] = False
        return cls._instances[cls]

    def _ensure_initialized(self, *args, **kwargs):
        """Ejecuta initialize(...) solo una vez por subclase."""
        c = self.__class__
        if not c._initialized.get(c, False):
            self.initialize(*args, **kwargs)
            c._initialized[c] = True

    @abstractmethod
    def initialize(self, *args, **kwargs):
        """Punto de inicialización definido por cada subclase."""
```

```
        pass


class ServicioConfiguracion(SingletonABC):
    """Ejemplo: servicio de configuración como Singleton."""

    def __init__(self, archivo=None):
        # Idempotente: solo inicializa una vez
        self._ensure_initialized(archivo=archivo)

    def initialize(self, archivo=None):
        import json
        self._settings = {}
        if archivo:
            try:
                with open(archivo, 'r') as f:
                    self._settings = json.load(f)
            except FileNotFoundError:
                print(f"Archivo {archivo} no encontrado; usando valores por defecto")

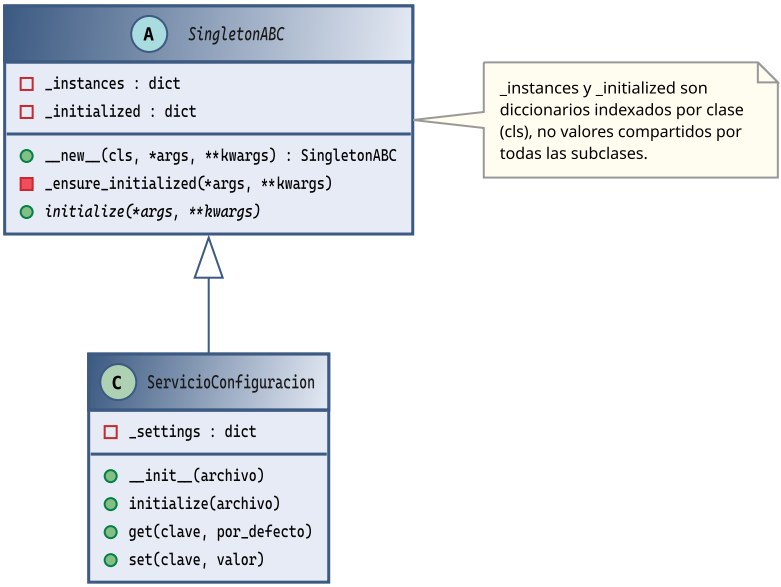
    def get(self, clave, por_defecto=None):
        return self._settings.get(clave, por_defecto)

    def set(self, clave, valor):
        self._settings[clave] = valor


# Uso
cfg1 = ServicioConfiguracion(archivo="config.json")
cfg2 = ServicioConfiguracion()

print(cfg1 is cfg2)  # True
cfg1.set('debug', True)
print(cfg2.get('debug'))  # True
```

Esta variante combina herencia (todas las subclases de `SingletonABC` obtienen el comportamiento Singleton en forma automática) con un contrato explícito (`initialize`) que cada subclase debe implementar. A diferencia del decorador, aquí sí hay una jerarquía de clases; a diferencia de la implementación GoF básica, el estado (`_instances`, `_initialized`) se indexa por subclase, permitiendo que cada clase derivada tenga su propia instancia única:



15.4.3 Comparación de enfoques

Cada enfoque tiene sus ventajas y casos de uso apropiados:

- **Decorador**: Reutilizable y no invasivo. Bueno cuando se necesita aplicar Singleton a múltiples clases sin modificar su estructura.
- **__new__ (GoF)**: Implementación tradicional. Clara y explícita, pero conviene controlar la inicialización para evitar costos repetidos en `__init__`.
- **Clase abstracta (ABC)**: Estandariza el contrato e integra *una sola instancia por subclase* con un hook de inicialización único. Útil cuando se desea guiar a equipos/estudiantes a seguir una interfaz común.

15.5 Consideraciones y críticas

Aunque el patrón Singleton es ampliamente utilizado, también es objeto de críticas y debe usarse con precaución.

15.5.1 Problemas potenciales

Acoplamiento global: El Singleton introduce un punto de acoplamiento global en la aplicación. Cualquier módulo que lo use depende de esa instancia global, lo que puede dificultar el testing y la refactorización.

Dificultad para testing: Los Singletons pueden complicar las pruebas unitarias porque mantienen estado entre tests. Esto puede causar que los tests se afecten mutuamente si no se limpia el estado correctamente.

Violación del principio de responsabilidad única: Un Singleton tiene dos responsabilidades: su funcionalidad principal y controlar su propia instanciación.

Problemas de concurrencia: En aplicaciones multithreading, se debe garantizar que la creación del Singleton sea thread-safe.

15.5.2 Alternativas y mejores prácticas

- Considerar inyección de dependencias en lugar de acceso global
- Usar Singleton solo cuando realmente se necesite una única instancia
- Preferir el enfoque de módulo en Python cuando sea posible
- Documentar claramente por qué se usa Singleton en cada caso
- Implementar métodos para resetear el estado en ambientes de testing

```python
# Ejemplo de Singleton con método de reset para testing
class TestableLogger:
    _instance = None
    _initialized = False

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super(TestableLogger, cls).__new__(cls)
        return cls._instance

    def __init__(self):
        if not TestableLogger._initialized:
            self.logs = []
            TestableLogger._initialized = True

    def log(self, message):
        self.logs.append(message)

    @classmethod
    def reset_for_testing(cls):
        """Resetea el Singleton para testing"""
```

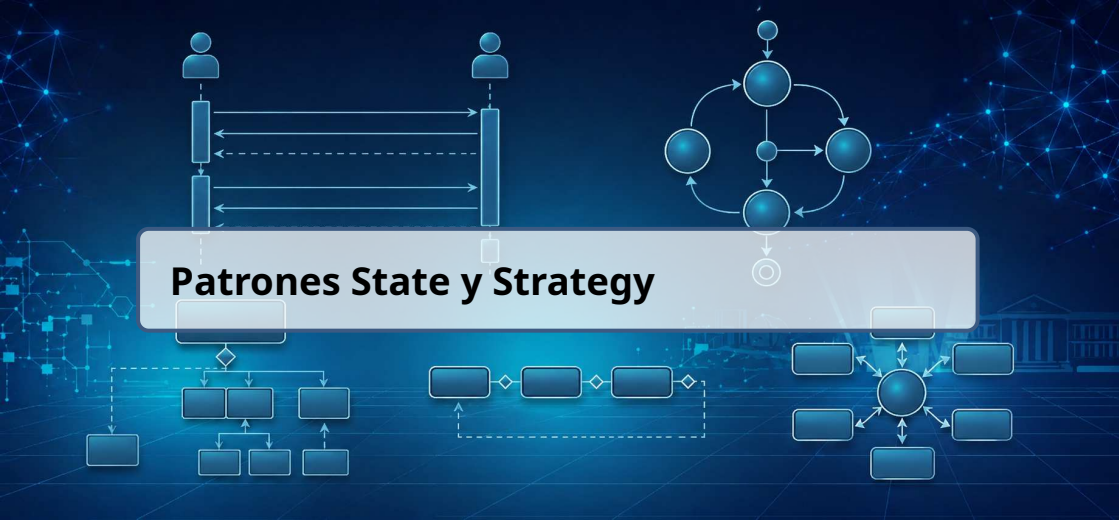
```
        if cls._instance is not None:
            cls._instance.logs = []
            cls._initialized = False
            cls._instance = None

# En los tests
logger = TestableLogger()
logger.log("Test message")
TestableLogger.reset_for_testing()
# Ahora el Singleton está limpio para el siguiente test
```

15.6 Conclusión

El patrón Singleton es una herramienta poderosa cuando se necesita garantizar una única instancia de una clase en todo el sistema. Python ofrece múltiples formas de implementarlo, desde la aproximación clásica con `__new__` hasta enfoques más pythónicos usando módulos o decoradores.

La elección del enfoque depende del contexto: para configuraciones simples, un módulo es suficiente; para casos más complejos con herencia o requisitos de thread-safety, las metaclases pueden ser más apropiadas. Lo importante es entender las implicaciones del patrón y usarlo juiciosamente, considerando alternativas como inyección de dependencias cuando el acoplamiento global pueda ser problemático.



16.1 Introducción

En este capítulo vamos a estudiar dos patrones de diseño del grupo de patrones comportamentales: *State* y *Strategy*. Ambos permiten cambiar el comportamiento de un objeto en tiempo de ejecución pero resuelven problemas distintos y con matices propios.

Primero motivaremos cada patrón, luego veremos su implementación en Python con ejemplos completos y, finalmente, propondremos ejercicios para consolidar los conceptos.

16.2 Patrón State

16.2.1 Motivación

El patrón State sirve cuando un objeto debe cambiar su comportamiento según su estado interno, y cuando ese cambio se puede modelar como la transición entre varias representaciones de estado. En lugar de escribir largas cadenas de condicionales (if/elif/else) que dependen de la variable de estado, el patrón State propone encapsular cada estado en una clase separada que implemente la interfaz común.

Ventajas:

- Reduce la complejidad ciclomática: evita condicionales anidados.
- Facilita añadir nuevos estados sin modificar la clase contexto.

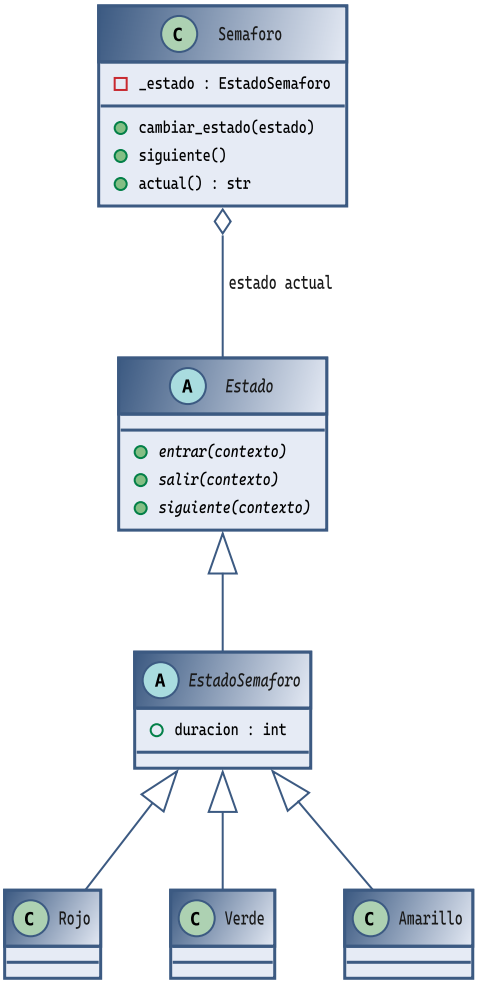
- Hace explícitas las transiciones de estado.

16.2.2 Contrato (interfaz)

El patrón define:

- Una clase *Context* que mantiene una referencia al estado actual y delega en él las operaciones.
- Una interfaz común para los *State* concretos que implementan el comportamiento correspondiente.

El siguiente diagrama de clases resume la estructura estándar de GoF aplicada al ejemplo del semáforo: el contexto (`Semaforo`) mantiene una referencia a un estado abstracto y delega en él; cada estado concreto conoce su transición al siguiente estado.



16.2.3 Ejemplo: semáforo

Motivación del ejemplo: un semáforo cambia entre estados (Rojo, Verde, Amarillo) y cada estado define la acción a realizar (por ejemplo, permitir o detener el paso). El ejemplo mostrará cómo encapsular los estados y las transiciones.

16.2.4 Código de ejemplo

Ejemplo: State (Semáforo)

```
"""
Ejemplo del patrón State: semáforo.

Contiene la implementación del contexto (Semaforo) y los estados (Rojo, Verde,
↪  Amarillo)
Incluye una función de simulación que espera con time.sleep al entrar en cada estado
y realiza la transición automáticamente para imitar un semáforo real.
"""
from abc import ABC, abstractmethod
import time
import sys
import select
import tty
import termios
try:
    import msvcrt
except Exception:
    msvcrt = None


class Estado(ABC):
    @abstractmethod
    def entrar(self, contexto):
        pass

    @abstractmethod
    def salir(self, contexto):
        pass

    @abstractmethod
    def siguiente(self, contexto):
        pass


class EstadoSemaforo(Estado):
    """Clase base para estados de semáforo que expone un atributo público `duracion`.

    Las subclases (Rojo, Verde, Amarillo) deberán sobrescribir `duracion`
    si necesitan un valor distinto.
    """
    # Duración por defecto en segundos para estados de semáforo
    duracion: int = 2


class Semaforo:
    def __init__(self, estado_inicial: EstadoSemaforo):
        # Atributo público interno que contiene el estado actual.
        # Tipado como EstadoSemaforo para que la simulación acceda a `duracion`.
        self._estado: EstadoSemaforo | None = None
        self.cambiar_estado(estado_inicial)

    def simular(self):
        """Ejecuta una simulación automática del semáforo hasta que el usuario pulse
        ↪  una tecla.

        La duración de cada estado se obtiene directamente desde el atributo
        público `duracion` del estado (clase derivada de `EstadoSemaforo`).
```

```
        """

        print("═══ Simulación: semáforo automático (pulse cualquier tecla para detener)
        ↪  ═══")
        while True:
            nombre = self.actual()
            estado_actual = self._estado
            dur = estado_actual.duracion
            print(f"Estado actual: {nombre} (esperando {dur} s)")
            try:
                time.sleep(dur)
            except KeyboardInterrupt:
                print('\nSimulación interrumpida por el usuario (Ctrl-C)')
                break
            # avanzar al siguiente estado
            self.siguiente()
        print("Simulación finalizada\n")

    def cambiar_estado(self, estado: EstadoSemaforo):
        if self._estado is not None:
            self._estado.salir(self)
        self._estado = estado
        self._estado.entrar(self)

    def siguiente(self):
        self._estado.siguiente(self)

    def actual(self):
        return type(self._estado).__name__


class Rojo(EstadoSemaforo):
    def entrar(self, contexto):
        print("Semáforo: ROJO - Detener vehículos")

    # Duración por defecto en segundos cuando el semáforo está en ROJO
    duracion = 3

    def salir(self, contexto):
        print("Saliendo de ROJO")

    def siguiente(self, contexto):
        contexto.cambiar_estado(Verde())


class Verde(EstadoSemaforo):
    def entrar(self, contexto):
        print("Semáforo: VERDE - Permitir paso")

    # Duración por defecto en segundos cuando el semáforo está en VERDE
    duracion = 4

    def salir(self, contexto):
        print("Saliendo de VERDE")

    def siguiente(self, contexto):
        contexto.cambiar_estado(Amarillo())


class Amarillo(EstadoSemaforo):
    def entrar(self, contexto):
```

```
        print("Semáforo: AMARILLO - Precaución")

    # Duración por defecto en segundos cuando el semáforo está en AMARILLO
    duracion = 2

    def salir(self, contexto):
        print("Saliendo de AMARILLO")

    def siguiente(self, contexto):
        contexto.cambiar_estado(Rojo())


def demo_semaforo():
    print("=== Demo: Semáforo (State) ===")
    sem = Semaforo(Rojo())
    for _ in range(6):
        print(f"Estado actual: {sem.actual()}")
        sem.siguiente()
    print()


def demo_simulacion():
    """Demo que ejecuta la simulación con duraciones razonables por estado."""
    sem = Semaforo(Rojo())
    # Usamos las duraciones definidas en cada estado.
    sem.simular()

if __name__ == '__main__':
    demo_simulacion()
```

Ejemplo: Strategy (Envíos)

```
"""
Ejemplo del patrón Strategy: cálculo de coste de envío.

Contiene implementaciones por clases y una versión sin herencia (diccionario de
↪  funciones).
"""
from abc import ABC, abstractmethod


class EstrategiaEnvio(ABC):
    @abstractmethod
    def calcular(self, paquete):
        pass


class Paquete:
    def __init__(self, peso_kg: float, distancia_km: float, dimensiones_cm=(0, 0, 0)):
        self.peso = peso_kg
        self.distancia = distancia_km
        self.dimensiones = dimensiones_cm


class EnvioEstandar(EstrategiaEnvio):
    def calcular(self, paquete: Paquete):
        base = 5.0
        por_kg = 1.0 * paquete.peso
```

```
        por_km = 0.05 * paquete.distancia
        total = base + por_kg + por_km
        return round(total, 2)


class EnvioExpreso(EstrategiaEnvio):
    def calcular(self, paquete: Paquete):
        base = 10.0
        por_kg = 2.0 * paquete.peso
        por_km = 0.12 * paquete.distancia
        total = base + por_kg + por_km
        return round(total, 2)


class TarifaPlana(EstrategiaEnvio):
    def calcular(self, paquete: Paquete):
        return 9.99


class CalculadorEnvio:
    def __init__(self, estrategia: EstrategiaEnvio):
        self._estrategia = estrategia

    def set_estrategia(self, estrategia: EstrategiaEnvio):
        self._estrategia = estrategia

    def calcular(self, paquete: Paquete):
        return self._estrategia.calcular(paquete)


def demo_strategy_clases():
    print("═══ Demo: Strategy con clases ═══")
    paquete = Paquete(peso_kg=2.5, distancia_km=120)
    calculador = CalculadorEnvio(EnvioEstandar())
    print("Estandar:", calculador.calcular(paquete))
    calculador.set_estrategia(EnvioExpreso())
    print("Expreso:", calculador.calcular(paquete))
    calculador.set_estrategia(TarifaPlana())
    print("Tarifa plana:", calculador.calcular(paquete))
    print()


def cliente_selecciona_estrategia_clase():
    print("═══ Demo: cliente selecciona estrategia (clases) ═══")
    paquete = Paquete(peso_kg=1.0, distancia_km=30)
    calculador = CalculadorEnvio(EnvioEstandar())
    print("Cliente eligió: estandar ->", calculador.calcular(paquete))
    calculador.set_estrategia(EnvioExpreso())
    print("Cliente cambió a: expreso ->", calculador.calcular(paquete))
    print()


# Implementación sin herencia (diccionario de funciones)
def estandar_fn(paquete):
    return round(5.0 + 1.0 * paquete.peso + 0.05 * paquete.distancia, 2)


def expreso_fn(paquete):
    return round(10.0 + 2.0 * paquete.peso + 0.12 * paquete.distancia, 2)
```

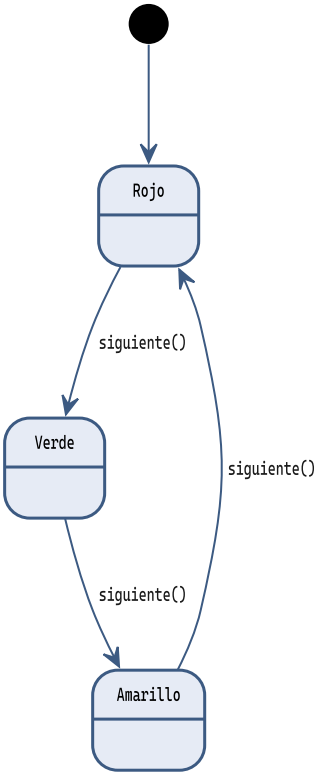
```
def demo_strategy_diccionario():
    print("== Demo: Strategy sin herencia (diccionario) ==")
    estrategias = {
        'estandar': estandar_fn,
        'expreso': expreso_fn,
        'plana': lambda p: 9.99,
    }
    paquete = Paquete(peso_kg=3.0, distancia_km=50)
    for nombre in ('estandar', 'expreso', 'plana'):
        coste = estrategias[nombre](paquete)
        print(f"{nombre}:", coste)
    print()


if __name__ == '__main__':
    demo_strategy_clases()
    cliente_selecciona_estrategia_clase()
    demo_strategy_diccionario()
```

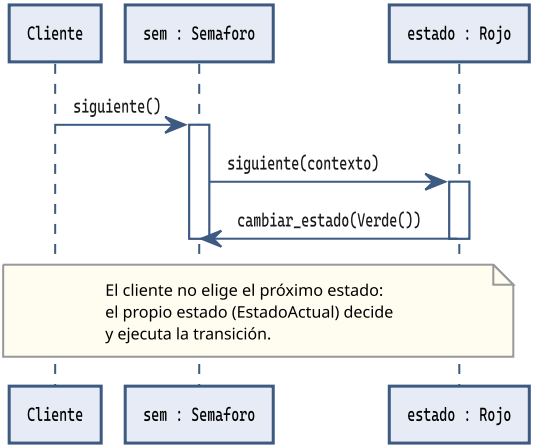
16.2.5 Comentarios de diseño

En el ejemplo del semáforo, la clase `Semaforo` actúa como *Context*. Las clases `Rojo`, `Verde` y `Amarillo` son los *State* concretos. Cada clase implementa los métodos necesarios y conoce la transición al siguiente estado. En el ejemplo de envío, `CalculadorEnvio` actúa como cliente del patrón Strategy y la estrategia concreta se inyecta desde fuera.

Más allá del diagrama de clases (que muestra la estructura estática heredada de GoF), conviene complementarlo con dos diagramas que reflejan aspectos que la estructura de clases no captura: el diagrama de estados, que muestra las transiciones reales entre estados; y el diagrama de secuencia, que muestra quién invoca a quién en tiempo de ejecución.



El diagrama de secuencia siguiente evidencia el punto clave del patrón State: el cliente le pide al contexto que avance (`siguiente()`), pero es el propio estado actual quien decide y ejecuta la transición, invocando `cambiar_estado(…)` sobre el contexto:



16.3 Patrón Strategy

16.3.1 Motivación

El patrón Strategy permite definir una familia de algoritmos intercambiables y hacer que el cliente los seleccione en tiempo de ejecución. Es útil cuando se desea variar el algoritmo de un objeto sin modificarlo, o cuando diferentes estrategias son necesarias según la configuración o el contexto.

Ventajas:

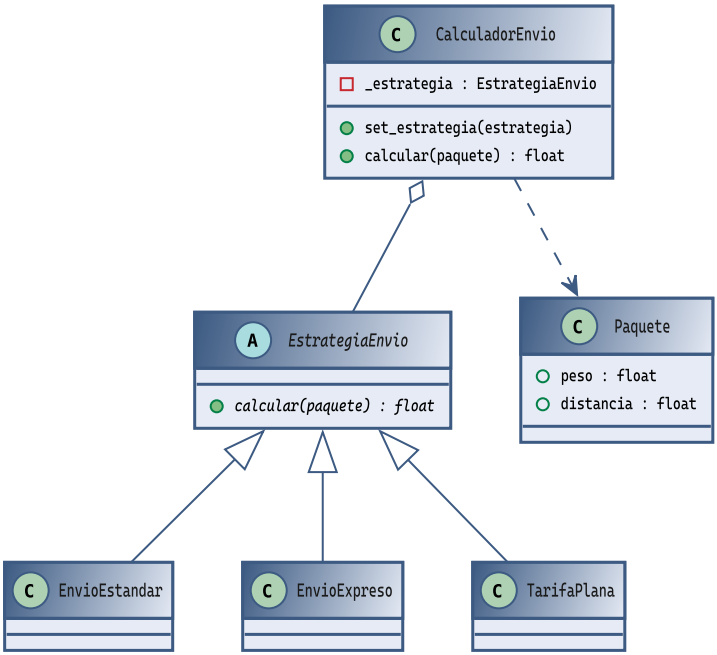
- Separa las políticas (algoritmos) de la lógica que las utiliza.
- Facilita pruebas unitarias de cada estrategia por separado.
- Añadir nuevas estrategias no requiere cambios en el cliente.

16.3.2 Contrato (interfaz)

- Una interfaz (por ejemplo, un método `execute(` … `)`) que implementan las estrategias concretas.
- Un objeto cliente que mantiene una referencia a la estrategia actual y la invoca.

El diagrama de clases del ejemplo de cálculo de envío es estructuralmente muy similar al de State (ambos usan una interfaz común y un contexto que delega), pero la relación semántica es distinta: aquí el contexto (`CalculadorEnvio`)

no conoce las transiciones entre estrategias, simplemente delega el cálculo a la estrategia que tiene asignada en cada momento.



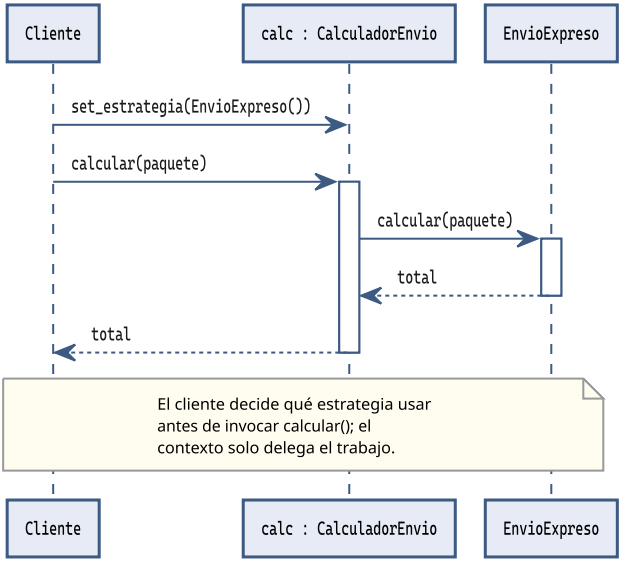
16.3.3 Ejemplo: cálculo de coste de envío

Motivación del ejemplo: un sistema de ventas puede calcular el coste de envío usando distintas políticas (envío estándar, exprés, gratuito por promoción, tarifa plana). El cliente (por ejemplo, un objeto `ShipmentCalculator`) delega el cálculo a la estrategia seleccionada.

16.3.4 Código de ejemplo

El mismo archivo de ejemplo contiene implementaciones de `Strategy` y una demostración de intercambio de estrategias en tiempo de ejecución.

El siguiente diagrama de secuencia complementa al diagrama de clases mostrando el contraste con State: aquí es el cliente quien decide y asigna la estrategia (`set_estrategia(` … `)`) antes de pedir el cálculo; el contexto no decide nada por sí mismo.



16.4 Ejercicios

1. Modificar el ejemplo del semáforo para que soporte modo peatonal: al activar el modo peatonal, las transiciones deben incorporar un estado adicional que permita el cruce peatonal.
2. Añadir una nueva estrategia de envío que calcule coste en función del peso y dimensiones y compararla con las estrategias existentes.
3. Diseñar pruebas unitarias (pytest) para las estrategias de envío: cubrir al menos la estrategia estándar y la exprés.
4. Reflexión: comparar State y Strategy; ¿en qué casos usarías uno u otro? Dar un ejemplo real donde ambos pueden aplicarse pero uno sea más adecuado.

16.5 Conclusión

State y Strategy son patrones primas hermanas: ambos encapsulan comportamientos intercambiables, pero State encapsula comportamientos que corresponden a estados del objeto y normalmente implican transiciones entre estados, mientras que Strategy encapsula algoritmos intercambiables que el

cliente selecciona sin cambiar su identidad.

Estos patrones fomentan la separación de responsabilidades, hacen el código más modular y facilitan la extensión del sistema.

16.5.1 Quién selecciona los cambios: una diferencia clave

Una distinción conceptual muy importante entre *State* y *Strategy* es *quién* decide qué comportamiento se usa en cada momento:

- **State: selección interna.** En el patrón State los cambios de comportamiento son el resultado de las transiciones internas del propio objeto contexto o de la lógica de los estados concretos. Cada estado conoce las reglas para pasar al siguiente estado o para modificar el comportamiento del contexto. El cliente que utiliza el contexto no selecciona directamente el estado; envía mensajes al contexto y este delega al estado actual. Es decir, el control del flujo de estados es responsabilidad del contexto/los estados.
- **Strategy: selección externa.** En el patrón Strategy la elección de la política (algoritmo) la realiza el cliente que usa el objeto contexto o una parte externa del sistema (por ejemplo, una fábrica, una configuración). El contexto delega el trabajo a la estrategia que tiene asignada, pero no decide por sí mismo qué estrategia usar; la estrategia se le inyecta desde fuera. Esto hace que Strategy sea ideal cuando la política depende de factores externos (configuración, preferencia del usuario, pruebas A/B, etc.).

En resumen: si el comportamiento debe cambiar porque el propio objeto evoluciona (transiciones internas), State es la opción natural; si el comportamiento cambia porque el cliente o la configuración lo decide, Strategy es la opción correcta.

16.5.2 Consecuencia en los ejemplos

En los ejemplos que acompañan este capítulo mostramos ambas situaciones:

- En la sección *State* (semáforo) el propio contexto y sus estados realizan las transiciones: el cliente no selecciona el siguiente estado, simplemente solicita operaciones (por ejemplo, `siguiente()`) y el contexto decide la transición.

- En la sección *Strategy* (envío) el cliente selecciona la estrategia: el ejemplo muestra cómo el cliente construye o cambia la estrategia del `CalculadorEnvio` y cómo, en la versión sin herencia, el cliente selecciona una función del diccionario de estrategias.

Esto tiene implicaciones de diseño: si quieres que la política pueda ser cambiada por el usuario o por la configuración, coloca la selección fuera del contexto (Strategy). Si en cambio la política depende del ciclo de vida o del estado interno del objeto, usa State.

16.6 Implementaciones sin herencia: cuándo usar un diccionario de funciones

En casos sencillos donde cada *State* o *Strategy* reduce su interfaz a un único método (por ejemplo `handle()` o `calculate(` … `)`), y donde las implementaciones no requieren mantener un estado interno complejo, es frecuente en Python usar alternativas más ligeras que las jerarquías de clases. Una opción idiomática y pragmática es emplear funciones o *callables* y almacenarlas en un diccionario que actúe como mapa de comportamiento.

Ventajas de esta aproximación:

- Menos código ceremonioso: no hay necesidad de definir muchas clases pequeñas.
- Más simple de leer cuando las funciones son pequeñas y puras.
- Fácil de serializar/configurar (por ejemplo, cargar el nombre de la estrategia desde configuración y buscarla en el diccionario).

Limitaciones:

- Menos flexible si las implementaciones necesitan conservar estado entre llamadas.
- Pierde la semántica explícita de una jerarquía de tipos (puede ser menos claro para ciertos diseños).

Ejemplo breve:

Ejemplo: Strategy con un diccionario de funciones

```
class Paquete:
    def __init__(self, peso, distancia):
        self.peso = peso
```

```
        self.distancia = distancia

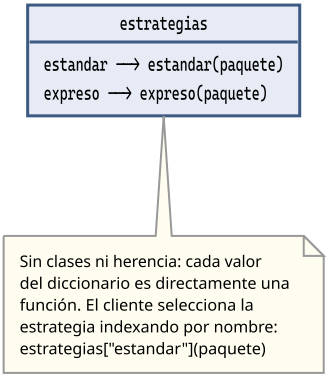

def estandar(paquete):
    return 5.0 + 1.0 * paquete.peso + 0.05 * paquete.distancia


def expreso(paquete):
    return 10.0 + 2.0 * paquete.peso + 0.12 * paquete.distancia


estrategias = {
    "estandar": estandar,
    "expreso": expreso,
}

paquete = Paquete(1.0, 100)
costo = estrategias["estandar"](paquete)
print(costo)
```

El siguiente diagrama complementario ilustra esta variante: no hay jerarquía de clases ni interfaz común, sino un objeto `dict` que asocia directamente un nombre de estrategia con la función (*callable*) que la implementa.



Esta técnica es perfectamente válida cuando el conjunto de operaciones es reducido y las alternativas son esencialmente funciones puras.